

Gebze Technical University
CSE 312 Spring 2024 Operating Systems
Assignment #02 Report

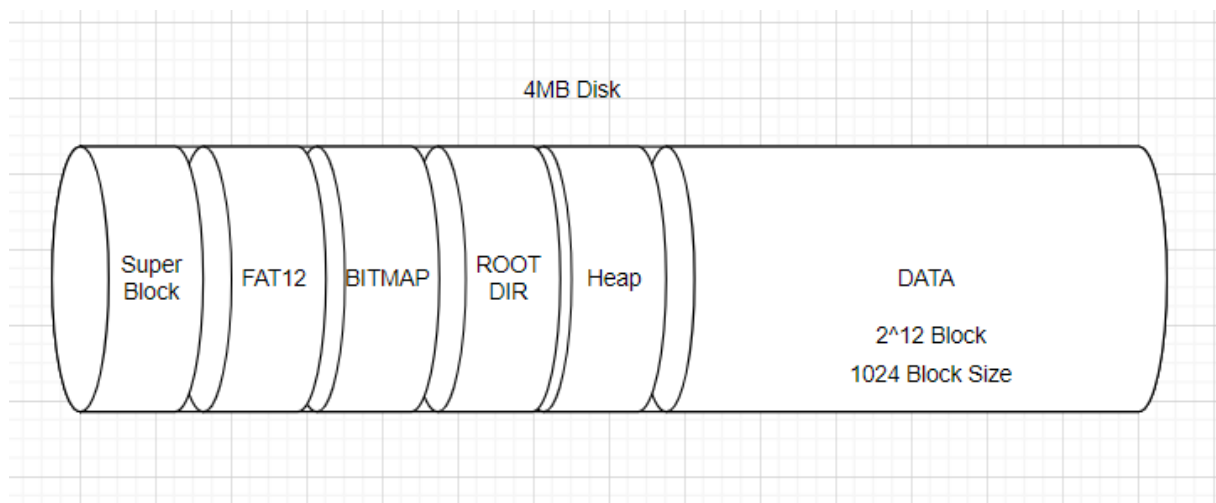
Abdulsamet ASLAN
200104004098

Project Overview

In this assignment, I implemented a simplified FAT12 file system in C++. The project is divided into three main parts, each incorporating essential fields and functionalities of the FAT12 file system.

Part 1: File System Design

My file system follows a structure similar to the image below. Instead of using i-nodes, it employs a FAT12 system. This approach leverages the FAT12 architecture that has 4096 blocks with 1024 or 512 block sizes each to manage files and directories.



```
SuperBlock superBlock;  
int FAT[NUM_BLOCKS];  
DirectoryEntry rootDir[NUM_BLOCKS];  
int bitmap[NUM_BLOCKS];  
std::vector<char*> block_storage;  
char heap[HEAP_SIZE];
```

Directory table and directory entries

Firstly, I created a directory table named rootdir with a specified number of blocks. Other than the root folder, folders hold their information in data block with heap. Heap is including filenames which are separated with “|” keyword. Each directory entry inside this array has the following attributes:

```
struct DirectoryEntry {  
    char *fileName;  
    int fileSize;  
    char owner_permissions;  
    time_t last_modification;  
    time_t creation_time;  
    char password[PASSWORD_SIZE];  
    int first_block;  
    bool isDirectory;  
    int file_count;  
};
```

File Attributes:

File Name: Supports any length.

Size: Stores the size of the file.

Owner Permissions: Read (R) and Write (W) permissions only.

Last Modification Date and Time.

File Creation Date and Time.

Password Protection: Optionally, files can be protected with a password.

Keeping the free blocks

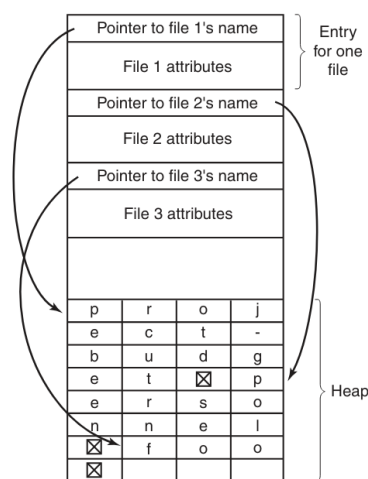
I implemented a bitmap approach to track free blocks. In this bitmap array, each element corresponds to a block in the actual storage. An element with a value of 1 indicates that the block is free, while a value of 0 signifies that the block is occupied.

1001101101101100
0110110111110111
1010110110110110
0110110110111011
1110111011101111
1101101010001111
0000111011010111
1011101101101111
1100100011101111
⋮
0111011101110111
1101111101110111

A bitmap

Arbitrary length of filenames:

To handle filenames of arbitrary lengths, each directory entry includes a filename pointer. When the program loads the filesystem, it sets the filenames by searching for them in the directory's heap. This approach ensures efficient management and dynamic allocation of memory for filenames. Each file name stored its parent directory heap separated by ']' character.



(b)

Permissions:

Each directory has an 8-byte password character array. If a password is set, the program compares the provided password with the actual one before allowing any operations.

Part 2: Creating File System

I implemented a function to create the filesystem file, which is always 4 MB in size. The function writes the superblock, FAT, bitmap, root directory, and heap for the root directory in that order. After writing these components, it calculates the remaining space for storage as 4 MB - (size of other parts). The remaining space is then allocated for block storage, divided into blocks based on the block size specified by the user. Additionally, the starting points for the superblock are initialized to be used in the next part of the implementation.

```
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$ ./makeFileSystem 1 mySystem.dat  
Initializing file system..
```



mySystem.dat

6.06.2024 23:13

DAT Dosyası

4.096 KB

```
superBlock.num_blocks = NUM_BLOCKS;  
superBlock.block_size = block_size;  
superBlock.bitmap_start = sizeof(superBlock);  
superBlock.fat_start = superBlock.bitmap_start + sizeof(bitmap);  
superBlock.root_dir_start = superBlock.fat_start + sizeof(FAT);  
superBlock.data_start = superBlock.root_dir_start + sizeof(rootDir) + sizeof(heap);  
superBlock.total_dir = 0;  
  
// Initialize block storage  
for (int i = 0; i < NUM_BLOCKS; i++) {  
    block_storage[i] = new char[block_size];  
    memset(block_storage[i], 0, block_size);  
}  
  
int usedBlocks = (sizeof(superBlock) + sizeof(bitmap) + sizeof(rootDir) + sizeof(FAT) + sizeof(heap)) / block_size;  
superBlock.dataBlockNum = NUM_BLOCKS - usedBlocks;  
  
save_filesystem();
```

Part 3: File System Operations

A program was developed to perform various file system operations on the created file system. The program supports the following operations: dir, mkdir, duple2fs, write, read, del, chmod, and addpw.

dir (List Directories):

This function mimics the ls command from Linux. **'list directories'** is implemented for this method. It navigates to the specified parent directory given in the path and prints the subfiles with their filename, size, creation time, modification time, permissions, and password protection status.

```
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$ ./makeFileSystem 1 mySystem.dat
Initializing file system..
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$ ./fileSystemOper mySystem.dat mkdir /mnt
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$ ./fileSystemOper mySystem.dat mkdir /ysa
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$ ./fileSystemOper mySystem.dat mkdir /temp
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$ ./fileSystemOper mySystem.dat dir /
```

Permissions	Size	Last Modification	Creation Time	Name
rw	0	Thu Jun 6 23:28:52	Thu Jun 6 23:28:52	mnt
rw	0	Thu Jun 6 23:28:54	Thu Jun 6 23:28:54	ysa
rw	0	Thu Jun 6 23:28:56	Thu Jun 6 23:28:56	temp

mkdir (Create directory):

This function creates a new directory entry with initial values and adds it to the desired folder as a subdirectory. The name of the new directory is also added to the heap of the parent directory. The **'create_directory'** function is used for this method. After adding the subdirectory to the corresponding parent folder, the data block of the parent folder is updated, since directories store their subdirectories in storage.

```
find_corr_dir(tokens, tokens.size() - 1, currentTable, fileIdx, last_first_block);

// add new directory to the last directory
DirectoryEntry newDir = DirectoryEntry();
newDir.fileName = (char*)tokens[tokens.size() - 1].c_str();
newDir.fileSize = 0;
newDir.owner_permissions = 'd';
newDir.last_modification = time(NULL);
newDir.creation_time = time(NULL);
newDir.first_block = find_free_block();
newDir.isDirectory = 1;
newDir.file_count = 0;
strcpy(newDir.password, "");

initialize_heap(newDir.first_block);

currentTable.push_back(newDir);

update_data_block(currentTable, last_first_block);

superBlock.total_dir += 1;
save_filesystem();
```

dumpe2Fs (Dump File System Information)

This function recursively traverses all the folders and prints the occupied blocks for each folder. Additionally, it prints all the information stored in the superblock, providing a comprehensive overview of the file system's structure and usage.

```
Directory: mnt
Block: 1
Block: 2
Block: 3
Directory: ysa
Block: 4
Block: 5
Block: 6
Directory: file
Block: 10
Block: 11
Block: 12
Directory: temp
Block: 7
Block: 8
Block: 9
Directory: file2
Block: 13
Block: 14
Block: 15
----- Super Block -----
Block Size: 1024
Number of Blocks: 4096
Bitmap Start: 32
FAT Start: 16416
Root Directory Start: 32800
Data Start: 296992
Data Block Number: 3806
Total Free Blocks: 4080
Total Directories: 5
----- Super Block -----
```

write (Write File):

This function is implemented in the **'write_file'** method. It first locates the corresponding parent folder and creates a new file within it. Then, it opens the specified file and copies its content to the newly created file by saving the contents to blocks on the disk. Additionally, it checks the permissions of the specified file and sets the permissions for the newly created file accordingly. It iterates the fat array to find next blocks.

```
int iterator = first_block;
for (int i = 0; i < num_blocks; i++) {
    fread(block_storage[iterator], 1, superBlock.block_size, file);
    if (i == num_blocks - 1) {
        FAT[iterator] = -1;
        break;
    } else {
        FAT[iterator] = find_free_block();
    }
    iterator = FAT[iterator];
}
```

```
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$ ./fileSystemOper mySystem.dat write /usr/ysa/file linuxFile.dat
rw574 bytes written
```

read (Read file):

This function is very similar to the write method. It opens the specified file and traverses the blocks of the corresponding file in the file system by using the FAT array. The contents of these blocks are then written to the specified file sequentially. Also file permissions of given file is also copied.

```
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$ ls -la linuxFile*
-rwxrwxrwx 1 samet samet 574 Jun  6 23:43 linuxFile.dat
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$ ./fileSystemOper mySystem.dat read /usr/ysa/filee linuxFileOut.dat
574 bytes read
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$ ls -la linuxFile*
-rwxrwxrwx 1 samet samet 574 Jun  6 23:43 linuxFile.dat
-rwxrwxrwx 1 samet samet 1024 Jun  6 23:57 linuxFileOut.dat
```

del (Delete File):

It again finds the parent folder of the given file, sets the FAT blocks to -1, and sets the bitmap blocks to 1. This means that the blocks used by this file are free for use. It also removes the file from the parent directory table.

```
void FileSystem::delete_file_blocks(int first_block){
    int iterator = first_block;
    while (FAT[iterator] != -1) {
        int next_iterator = FAT[iterator];
        bitmap[iterator] = 1;
        FAT[iterator] = -1;
        iterator = next_iterator;
    }
    bitmap[iterator] = 1;
    FAT[iterator] = -1;
}
```

```
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$ ./fileSystemOper mySystem.dat mkdir /usr
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$ ./fileSystemOper mySystem.dat mkdir /usr/ysa
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$ ./fileSystemOper mySystem.dat mkdir /usr/ysa/dir
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$ ./fileSystemOper mySystem.dat write usr/ysa/file linuxFile.dat
rw574 bytes written
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$ ./fileSystemOper mySystem.dat dir usr/ysa
Permissions  Size      Last Modification      Creation Time      Name
-----
rw          0      Fri Jun  7 00:18:07      Fri Jun  7 00:18:07      dir
rw         574      Fri Jun  7 00:18:13      Fri Jun  7 00:18:13      file
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$ ./fileSystemOper mySystem.dat del usr/ysa/file
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$ ./fileSystemOper mySystem.dat dir usr/ysa
Permissions  Size      Last Modification      Creation Time      Name
-----
rw          0      Fri Jun  7 00:18:07      Fri Jun  7 00:18:07      dir
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$
```

chmod (Change Permission):

This command changes the file permissions using the specified permission command. Acceptable commands are “r”, “w”, and “rw”. After the file permissions are changed, other processes check if the permissions allow access.

```
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$ ./fileSystemOper mySystem.dat dir usr/ysa
Permissions  Size      Last Modification      Creation Time      Name
-----
rw           0        Fri Jun 7 00:18:07      Fri Jun 7 00:18:07      dir
rw           0        Fri Jun 7 00:20:19      Fri Jun 7 00:20:19      dir2
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$ ./fileSystemOper mySystem.dat chmod usr/ysa/dir r
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$ ./fileSystemOper mySystem.dat dir usr/ysa
Permissions  Size      Last Modification      Creation Time      Name
-----
r            0        Fri Jun 7 00:22:04      Fri Jun 7 00:18:07      dir
rw           0        Fri Jun 7 00:20:19      Fri Jun 7 00:20:19      dir2
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$ █
```

addpw (Add Password):

This command secures the given file with a password. After adding the password, the 'read' command prompts for the password. If the password is not provided or is incorrect, an error message is displayed.

```
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$ ./fileSystemOper mySystem.dat dir usr/ysa
Permissions  Size      Last Modification      Creation Time      Name
-----
rw           0        Fri Jun 7 00:27:37      Fri Jun 7 00:27:37      dir
rw          574        Fri Jun 7 00:27:47      Fri Jun 7 00:27:47      file
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$ ./fileSystemOper mySystem.dat addpw usr/ysa/file 1234
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$ ./fileSystemOper mySystem.dat dir usr/ysa
Permissions  Size      Last Modification      Creation Time      Name
-----
rw           0        Fri Jun 7 00:27:37      Fri Jun 7 00:27:37      dir
rw          574        Fri Jun 7 00:28:07      Fri Jun 7 00:27:47      file [Password protected]
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$ ./fileSystemOper mySystem.dat read usr/ysa/file linuxFileOut.dat
Password incorrect
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$ ./fileSystemOper mySystem.dat read usr/ysa/file linuxFileOut.dat 1234
Password correct
574 bytes read
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$ █
```

Final Test Results:

```
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$ ./makeFileSystem 1 mySystem.dat
Initializing file system..
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$ ./fileSystemOper mySystem.dat mkdir /usr
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$ ./fileSystemOper mySystem.dat mkdir /usr/ysa
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$ ./fileSystemOper mySystem.dat mkdir /bin/ysa
Directory does not exist
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$ ./fileSystemOper mySystem.dat write /usr/ysa/file1 linuxFile.data
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$ ./fileSystemOper mySystem.dat write /usr/file2 linuxFile.data
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$ ./fileSystemOper mySystem.dat write /file3 linuxFile.data
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$ ./fileSystemOper mySystem.dat dir /
Permissions  Size      Last Modification      Creation Time      Name
-----
rw          0        Fri Jun 7 20:20:22      Fri Jun 7 20:20:22      usr
rw         574        Fri Jun 7 20:21:29      Fri Jun 7 20:21:29      file3
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$
```

```
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$ ./fileSystemOper mySystem.dat del /usr/ysa/file1
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$ ./fileSystemOper mySystem.dat dumpt2fs
Directory: usr
Block: 1
Block: 2
Block: 3
Directory: ysa
Block: 4
Block: 5
Block: 6
Directory: file2
Block: 8
Directory: file3
Block: 9
Directory: ysa
Block: 4
Block: 5
Block: 6
Directory: file2
Block: 8
|----- Super Block -----|
| Block Size: 1024           |
| Number of Blocks: 4096    |
| Bitmap Start: 32          |
| FAT Start: 16416          |
| Root Directory Start: 32800|
| Data Start: 296992        |
| Data Block Number: 3806   |
| Total Free Blocks: 4087   |
| Total Directories: 4      |
|----- Super Block -----|
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$
```

```
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$ ./fileSystemOper mySystem.dat read /usr/file2 linuxFile2.data
574 bytes read
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$ cmp linuxFile.data linuxFile2.data
cmp: EOF on linuxFile.data after byte 574, in line 1
```

```
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$ ./fileSystemOper mySystem.dat chmod /usr/file2 w
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$ ./fileSystemOper mySystem.dat read /usr/file2 linuxFile2.data
Permission denied
```

```
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$ ./fileSystemOper mySystem.dat addpw /usr/file2 test123
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$ ./fileSystemOper mySystem.dat read /usr/file2 linuxFile2.data
Password incorrect
samet@Samet:/mnt/d/GTU/OS/hw2/src_cpp$ ./fileSystemOper mySystem.dat read /usr/file2 linuxFile2.data test123
Password correct
574 bytes read
```